# Guía del paquete de reproducibilidad (para Mónica)

Esta guía explica, en palabras simples, qué hay en la carpeta `reproducibility/`, para qué sirve cada programa, cómo se ejecuta y cómo subirlo a Zenodo. El manual oficial, en inglés, para editores y revisores es `reproducibility/README.md`; esta guía es para que tú puedas entenderlo y defenderlo.

---

## 1. Qué es el paquete

Es una carpeta autocontenida con todo lo que respalda el paper:

| Carpeta | Qué contiene | En palabras simples |
|---|---|---|
| `analysis/` | los programas del método y de cada análisis (Python) | lo que "calcula" |
| `results/` | los resultados de esos cálculos, un archivo por problema (unos 40 MB) | los números del paper |
| `figures_tables/` | los programas que hacen cada figura, tabla y nota del suplemento | lo que "dibuja" |
| `lean_proofs/` | las demostraciones en Lean 4 (32 módulos, 309 teoremas y lemas, 28 modelos, y los testigos de no vacuidad) | lo que "demuestra" |
| `lean_translation/` | el traductor SBML → Lean y la comprobación de la traducción | el puente entre modelos y Lean |
| `scripts/` | descarga del benchmark y comparación de resultados | utilidades |

No incluye: los problemas PEtab (se descargan con `scripts/get_benchmark.sh`) ni los datos de pasos del participante del modelo SCT (se entregan "a pedido razonable", como dice el paper).

---

## 2. Qué hace cada programa (en orden de uso)

| Programa (`analysis/`) | Qué hace | Produce |
|---|---|---|
| `config.py` | guarda todos los umbrales del paper ($\delta$ = 1 %, $\tau_R$ = 0,89, $\kappa$/VIF ≤ 10, $\tau_e$ = 0,436, 15 escenarios, semilla 42) | — |
| `petab_problem.py` | simula las **salidas medidas** de un problema PEtab: $y_i = h(g_i(x(t_i), \theta))/\sigma_i$ | — |
| `simulator_check.py` | compara nuestra simulación con la del benchmark | Tabla S9 |
| `selection.py` | **el método**: sensibilidad J, etapa 1 (energía), etapa 2 (efecto finito), $\kappa$/VIF, admisibilidad con e_fit, poda | Fig. 2, Tabla 1, Tabla S1 |
| `selection.py --variant …` | lo mismo cambiando un ajuste ($\delta$, tolerancias, ±1 %/±10 %, $\tau_R$, $\kappa$/VIF, $\tau_e$) | Fig. 5, Tabla S14 |
| `external_validation.py` | ajusta con unos experimentos y predice los otros (datos reales y sintéticos) | Fig. 3, Fig. S1, Tabla S12 |

| Programa (analysis/) | Qué hace | Produce |
|---|---|---|
| synthetic_seeds.py | experimento sintético con 20 semillas; con SINT_SOLO_SENAL=1 calcula solo la señal y(θ*)−y($\theta_0$) en σ, y con SINT_AMPLITUD=0.05/0.50 repite el experimento con otras amplitudes (punto C3) | Fig. 3b; tabla y figura de señal y amplitudes |
| robustness.py | mide el error a ±1…50 % con las mismas 15 direcciones; patrones A–D | Fig. 5, Tablas S10–S11, Fig. S2 |
| robustness.py con ROBUST_INDEP=1 | lo mismo con direcciones **nuevas** en cada tamaño (punto B5 de la auditoría) | tabla y figura de direcciones independientes (nota de robustez) |
| absolute_error.py | expresa el error que deja el reducido en unidades del ruido σ (punto C1) | tabla del error en σ (nota de ajustes) |
| single_parameter.py | prueba si un solo parámetro bastaría (justifica k_min = 2) | Tabla S13 |
| crauste_blowup.py | muestra por qué Crauste no tiene reducción local | texto de Resultados |
| timing.py | mide el tiempo de una simulación | Tabla S15 |
| sct_model.py, sct_analysis.py | el ejemplo SCT (necesita los datos del participante) | Tabla S16, Fig. S3 |

| Programa (lean_translation/) | Qué hace |
|---|---|
| sbml_to_lean.py | traduce cada modelo SBML a un archivo Lean con números racionales exactos |
| verify_translation.py | comprueba, sin usar el traductor, que el archivo Lean es el mismo modelo que el SBML (diferencia ∼$10^{-12}$) |
| mutation_control.py | control: si se cambia un número del archivo Lean en 1 %, la comprobación lo detecta (∼$10^{-2}$) |

figures_tables/build_all.py ejecuta todos los generadores de figuras, tablas y notas; entre ellos make_review_notes.py, que produce las tablas y figuras de los análisis adicionales (direcciones independientes, señal y amplitudes, error en σ y variantes de ajustes de Lang y Raimundez), y make_timing_notes.py, que reúne el tiempo de cómputo de cada análisis en cada problema (campo segundos de los resultados) y produce las tablas de la nota "Computing time of the analyses" del suplemento; su objetivo es mostrar que seleccionar y verificar un subconjunto es asequible (minutos en casi todos los problemas, mucho menos que la verosimilitud perfilada del modelo completo, que no fue viable en los problemas grandes). figures_tables/cifras_texto.py recalcula las cifras que cita el texto principal (admisibles, identificabilidad, validación, comparación de criterios, robustez) para comprobarlas tras cualquier cambio.

**2b. Programas añadidos en la segunda auditoría (versión 1.5)**

| Programa | Qué hace | Por qué | Produce |
|---|---|---|---|
| selection.py con REPRO_FIXED_SCEN=1 | repite la selección con un conjunto fijo de escenarios y registra la monotonía | antes un escenario que fallaba se descartaba y cada subconjunto podía evaluarse con escenarios distintos; el Teorema 2 suponía monotonía | results/selection_fixedscen/ (ningún subconjunto cambió; 0 violaciones en 660 pares) |
| selection.py con REPRO_OUTPUTS=train | repite tu selección viendo solo las salidas de entrenamiento | los demás criterios elegían con entrenamiento y el tuyo con todas las salidas | results/selection_train_fixedscen |

| Programa | Qué hace | Por qué | Produce |
|---|---|---|---|
| `qr_selection.py` con `QR_VARIANT=gks` | QR pivotado estándar (Golub–Klema–Stewart) | el QR anterior no era el algoritmo de la literatura | `results/qr_selection_gks/` |
| `criteria_tests.py` | admisibilidad de todos los criterios, de k = 0 y de 20 subconjuntos al azar; con `SINT_FAMILIA=loguniforme` o `dispersa`, otras familias de verdades sintéticas | controles nulos y una prueba sintética que no use la misma familia que tu método | `results/criteria_tests_*` |
| `efit_nonlinear.py` | re-estimación no lineal de los 6 problemas de primer orden | su admisibilidad se había decidido solo con la aproximación lineal | `results/efit_nonlinear/` (los 6 se confirman) |
| `conditioning_check.py` | κ y VIF con columnas centradas y sin centrar | el código y Lean usan normalizaciones distintas | `results/conditioning_check.json` |
| `identifiability_profiles.py` con `ID_DENSE=1` o `ID_SUBSETS=…` | perfiles con 20 puntos por lado, o de otros subconjuntos | ver si la clasificación depende de la resolución, y perfilar los subconjuntos de entrenamiento | `results/identifiability/profiles_densa/` o `profiles_selection_train_fixedscen/` |
| `identifiability_box.py` | comprueba las hipótesis de los teoremas de Lean de identificabilidad en toda la región alrededor del óptimo (factor 10, 5, 3, 2 y 1,5) | antes se comprobaban solo sobre el perfil | `results/identifiability/box/`, `results/identifiability/audit2/` |
| `scripts/fase_a_audit2.sh`, `scripts/identificabilidad_auditoria.sh` | lanzan todo lo anterior en una sola vez, de muchos núcleos | — | — |
| `figures_tables/audit2_stats.py`, `audit2_tables.py` | estadísticas (McNemar, bootstrap, Holm), Tabla 2 y tablas nuevas del suplemento | — | Tabla 2, Fig. 4, tablas S nuevas |
| `lean_proofs/NonVacuity.lean` | ejemplos verificados que muestran que las hipótesis de cada teorema central se pueden cumplir | descartar teoremas vacíos | — |
| `lean_proofs/DEFINITIONS_AUDIT.md` | cotejo cada definición de Lean con el paper y el código | descartar que el enunciado no diga lo que el paper afirma | — |

## 3. Cómo ejecutarlo en tu computadora

Necesitas Python 3.11. En una terminal, dentro de la carpeta `reproducibility/`:

```
python -m venv .venv
.venv\Scripts\activate          (en Windows; en Mac/Linux: source .venv/bin/activate)
pip install -r requirements.txt
```

**Nivel A (unos dos minutos): rehacer figuras y tablas desde los resultados guardados**

```
cd figures_tables
python build_all.py
```

Aparecen en `figures_tables/output/`. Para Fig. 6 y las copias EPS/TIFF hacen falta `pdflatex` y poppler (`pdftops`, `pdftoppm`).

**Nivel B (20 minutos con muchos núcleos, 1–2 horas en un portátil): compilar Lean**

Instalar elan (https://github.com/leanprover/elan) y luego, dentro de `lean_proofs/`: `./verify.sh` (en Windows, desde Git Bash o WSL).

**Nivel C (1–2 días en total): recalcular desde cero**

```
bash scripts/get_benchmark.sh
set REPRO_RESULTS=%CD%\results_rerun    (Windows cmd; en PowerShell: $env:REPRO_RESULTS="$PWD\results_rerun"
cd analysis
python selection.py Boehm_JProteomeRes2014
cd ..
python scripts/compare_results.py results_rerun/selection results/selection
```

Puedes recalcular un solo problema (como en el ejemplo) o todos (sin nombres). Los pasos y tiempos están en la tabla del README.

**Análisis de la segunda auditoría (en un servidor Linux de muchos núcleos):**

```
bash scripts/get_benchmark.sh
NCORES=88 nohup bash scripts/fase_a_audit2.sh > fase_a.log 2>&1 &        (unas 6 h con 88 núcleos)
NPROC=30 nohup bash scripts/identificabilidad_audit2.sh > identif.log 2>&1 &  (unas 5 h)
cd analysis && python conditioning_check.py
```

Al terminar, `cd figures_tables && python build_all.py` rehace la Tabla 2, la Fig. 4 y las tablas nuevas.

---

## 4. Qué comprobé antes de entregártelo

- **Código equivalente.** Los programas del paquete son versiones ordenadas (en inglés) de los que usamos (`certificados/`). Comprobé automáticamente que la lógica es la misma, comparando la estructura del código función por función, y además volví a ejecutarlos y comparé los resultados con los guardados (ver la sección 6).
- **Traducción a Lean.** Al volver a ejecutar el traductor, los 28 archivos de modelos salen **idénticos** a los que se compilaron.
- **Figuras y tablas.** En un entorno de Python nuevo, instalado solo con `requirements.txt`, el nivel A regenera todo en unos 20 segundos, y las tablas, las figuras y las notas coinciden con las del paper.
- **Benchmark.** `get_benchmark.sh` descarga exactamente los mismos archivos que usamos (commit fcbddf1).
- **Segunda auditoría (16-10-2026).** El nivel A regenera en Windows todas las tablas del paper y del suplemento **idénticas** a las enviadas (solo falta la nota SCT, que necesita los datos del participante). Algunas notas del suplemento se retocaron a mano después de generarlas (redacción y números de figura), así que su texto regenerado puede variar en la redacción, no en las cifras.
- **Lean (16-10-2026).** Compilación desde cero en una copia limpia: 309 de 309 teoremas y 28 de 28 modelos solo con los axiomas estándar; `lean4checker` aceptó todos los módulos; un control negativo con `sorry` fue detectado. Se corrigieron dos errores de `verify.sh` (no comprobaba de verdad los modelos y fallaba al ejecutarlo dos veces).
- **Un error mío que corregí al armar el paquete.** Los certificados nuevos de identificabilidad se guardaban en la misma carpeta que leen las tablas del paper y las alteraban al regenerarlas; ahora están en `results/identifiability/audit2/` y las tablas regeneradas coinciden con las del paper.

---

## 5. Correcciones que salieron al armar el paquete

Al rehacer todo con cuidado aparecieron cuatro cosas que corregí en el paper:

1. **Prueba de un solo parámetro.** La validación externa del mejor parámetro único no reajustaba los parámetros de calibración (escalas, offsets), mientras que la del subconjunto S sí. Lo recalculé de forma justa (con la calibración, igual que S) y actualicé la Tabla S13 y la frase del paper.
2. **Crauste.** El paper decía que con ρ_E −1 % los efectores pasan de $5\times10^2$ a $5\times10^5$ en cuatro días, pero eso también ocurre con los parámetros publicados. Lo correcto: con los parámetros publicados los efectores llegan a $\sim10^6$ cerca del día 10 y luego bajan; con un cambio del 1 % siguen creciendo y la solución se rompe entre los días 8,8 y 9,9. Ocurre en 9 de las 24 perturbaciones (antes decía “8–9”) y en 7 de 15 escenarios, con cualquier tolerancia.

3. **Robustez: una sola corrida.** Había dos corridas de robustez (la segunda guarda cada escenario). Ahora todo sale de la segunda. Solo cambia Chen, que en Methods pasa de "entre 7 y 13 escenarios" a "entre 8 y 13"; los patrones y todas las demás cifras son iguales.
4. **Rutas.** El paper y el suplemento citaban programas de `certificados/`; ahora citan el paquete.

---

## 6. Cómo subirlo a Zenodo (paso a paso)

1. Entra a https://zenodo.org con tu cuenta (puedes usar ORCID o GitHub).
2. **New upload**.
3. Arrastra el archivo `Paquete_reproducibilidad_v1.5.0_2026-10-16.zip` (está en tu carpeta de Descargas y en `zenodo/` del repositorio). Como el registro ya existe (DOI 10.5281/zenodo.23338175), **no crees uno nuevo**: entra al registro y pulsa **New version**; Zenodo le da un DOI nuevo a la versión y mantiene el historial.
4. Pulsa **Reserve DOI** (o "Get a DOI now"): Zenodo te muestra el DOI, algo como `10.5281/zenodo.1234567`.
5. Completa los datos:
   - *Resource type*: Software.
   - *Title*: "Reproducibility package: Few biological parameters suffice to reproduce measured dynamics in systems biology models".
   - *Creators*: los cuatro autores, con afiliación ESPOL (y ORCID si los tienen).
   - *Description*: puedes copiar el primer párrafo del README.
   - *License*: MIT.
   - *Version*: 1.5.0. El título debe ser **idéntico** al del paper (punto N9 de la auditoría).
6. **Antes de publicar**, pon el DOI en el paper. En `main.tex` y `supplementary.tex` (arriba del todo) reemplaza `10.5281/zenodo.XXXXXXX` por tu DOI en la línea `\newcommand{\zenododoi}{...}`; con eso se actualizan todas las menciones. Compila de nuevo.
7. Publica en Zenodo. Si después hay cambios (por ejemplo tras la revisión), se sube una **nueva versión** del mismo registro y Zenodo le da otro DOI, manteniendo el historial.

Si prefieres no publicar todavía, Zenodo permite reservar el DOI y dejar el registro en borrador.

---

## 7. Si un revisor pregunta…

| Pregunta | Respuesta y dónde está |
|---|---|
| ¿Cómo reproduzco una cifra? | README, tabla "Where each result comes from": cifra → programa → archivo de resultados |
| ¿Qué significa un campo del JSON? | `DATA_DICTIONARY.md` |
| ¿Son deterministas los resultados? | Sí: semillas fijas. Solo Chen puede variar levemente (algunas simulaciones fallan al integrar) |
| ¿Por qué hay nombres en español en los resultados? | Son los nombres originales de los programas; el diccionario los traduce |
| ¿El archivo Lean es de verdad el modelo del benchmark? | `verify_translation.py` (diferencia $\sim 10^{-12}$) y el control del 1 % |
| ¿Por qué no está el modelo completo de Chen y Raimundez en el sintético? | Tienen más de 100 parámetros libres; con datos reales sí se reajusta (validación anidada) |
| ¿Por qué Lang no es admisible? | La búsqueda voraz no llega a un par admisible con los ajustes por defecto, aunque existen (nota *Random scenarios and large problems*, tabla de variantes de Lang) |
| ¿Cómo se tratan los parámetros negativos o nulos en los escenarios? | Perturbación multiplicativa sin recorte: conservan el signo; los nulos se representan por $10^{-12}$ y nunca se eligen (misma nota) |
| ¿Se obtiene lo mismo en otra máquina? | Sí: la selección de Lang y Raimundez coincidió con libRoadRunner 2.9.2 y 2.10.0 (e_fit iguales hasta $10^{-8}$) |
| ¿Dónde están los datos del SCT? | No son públicos; se entregan a pedido razonable |

| Pregunta | Respuesta y dónde está |
|---|---|
| ¿Tu método vio los datos de prueba al elegir? | Sí en la selección principal; por eso se repitió con solo entrenamiento (`selection_train_fixedscen/`): rinde como Brun y mejor que QR y perfil |
| ¿Pasar la prueba de admisibilidad tiene mérito? | Controles nulos: al azar ~16/28, solo calibración 9/28, frente a 28/28 (`criteria_tests_fixedscen/`) |
| ¿Las demostraciones de Lean son vacías? | No: `NonVacuity.lean` da un ejemplo verificado de cada teorema central; `DEFINITIONS_AUDIT.md` compara definiciones |
| ¿Cómo sé que Lean no dejó pasar un hueco? | `verify.sh` informa los axiomas de los 309 teoremas y 28 modelos, `lean4checker` los revisa otra vez, y un `sorry` de prueba se detecta (`BUILD_LOG.md`) |